

From LLMs to Agentic AI

LANGCHAIN

Understanding the foundations before building AI applications

What Are Generative AI, LLM and LangChain?



GENERATIVE AI

A type of AI that can create new content such as text, images, audio, video and code based on patterns learned from data.

- ChatGPT
- Image generation
- Code generation
- AI music
- AI video



LLM

Large Language Model

An AI model trained on large amounts of text and code to understand and generate human-like language.

- GPT · Gemini · Claude
- Llama · Mistral

Understand → Reason → Generate → Transform



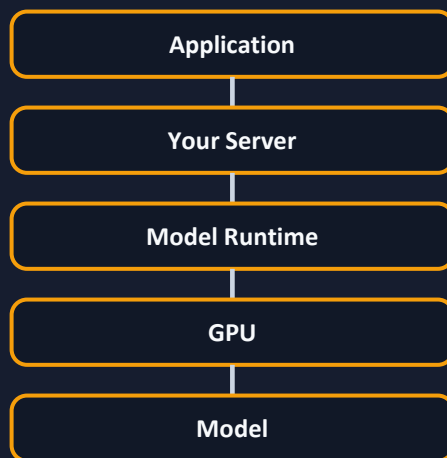
LANGCHAIN

An open-source framework for building applications powered by language models — connecting models with prompts, tools, data, retrieval and application logic.

LLM + Context + Tools + Data + Logic

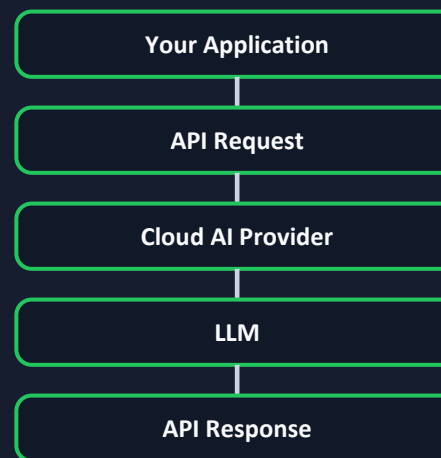
How LLM APIs Solved Part of the Problem

BEFORE — SELF-HOSTED MODEL



- GPU infrastructure
- Model storage & memory
- Scaling & deployment
- Monitoring & updates
- High infrastructure cost

NOW — API-BASED MODEL ACCESS



- No model hosting required
- No GPU management
- Easier integration
- Scalable infrastructure
- Multiple model choices

The application does not need to host the model itself — it communicates with a model through an API.

API solves: "How do I access the model?"

LangChain helps solve: "How do I build an application around the model?"

Problems Solved by LangChain

Before LangChain, if we wanted to build an AI application using different LLM providers such as **OpenAI**, **Google Gemini**, or **Anthropic Claude**, we generally had to learn and maintain a different SDK/API integration for each provider.

1 Open Ai

```
from openai import OpenAI

client = OpenAI()

response =
client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "user",
"content": "Explain Python"}
    ]
)

print(response.choices[0].message.content)
```

2 Google Gemini

```
from google import genai

client = genai.Client()

response =
client.models.generate_content(
    model="gemini-2.0-flash",
    contents="Explain Python"
)

print(response.text)
```

3 Anthropic Claude

```
from anthropic import Anthropic

client = Anthropic()

response = client.messages.create(
    model="claude-3-5-sonnet-latest",
    max_tokens=500,
    messages=[
        {"role": "user", "content":
"Explain Python"}
    ]
)

print(response.content[0].text)
```

Problems Solved by LangChain

1 Open Ai - langchain

```
from langchain_openai import
ChatOpenAI

model = ChatOpenAI(
    model="gpt-4o-mini",
    api_key="OPENAI_API_KEY"
)

response = model.invoke("What is
Python?")

print(response.content)
```

2 Google Gemini

```
from langchain_google_genai
import ChatGoogleGenerativeAI

model = ChatGoogleGenerativeAI(
    model="gemini-2.0-flash",
    google_api_key="GEMINI_API_
KEY"
)

response = model.invoke("What
is Python?")

print(response.content)
```

3 Anthropic Claude

```
from langchain_anthropic import
ChatAnthropic

model = ChatAnthropic(
    model="claude-3-5-sonnet-latest",
    api_key="CLAUDE_API_KEY"
)

response = model.invoke("What is
Python?")

print(response.content)
```

WHY USE LANGCHAIN?

Benefits and Use Cases of LangChain



WHERE IS LANGCHAIN USED?

- Customer support
- Document Q&A
- Research assistants
- Coding assistants
- Business automation
- Knowledge assistants
- Data analysis
- AI agents
- Enterprise AI apps

LangChain is most useful when an LLM needs to become part of a larger application.

What Can You Build with LangChain?

1

AI Chatbot

Conversational assistant grounded in your rules.

2

Chat with PDF

Ask questions about your uploaded documents.

3

Knowledge Assistant

Answers grounded in your own knowledge base.

4

Research Assistant

Search, retrieve, analyze and summarize.

5

Customer Support Agent

Answer questions and use business tools.

6

AI Coding Assistant

Understand, generate and explain code.

7

Document Analyzer

Extract insight from large document sets.

8

Data / SQL Assistant

Turn questions into data answers.

9

AI Agent with Tools

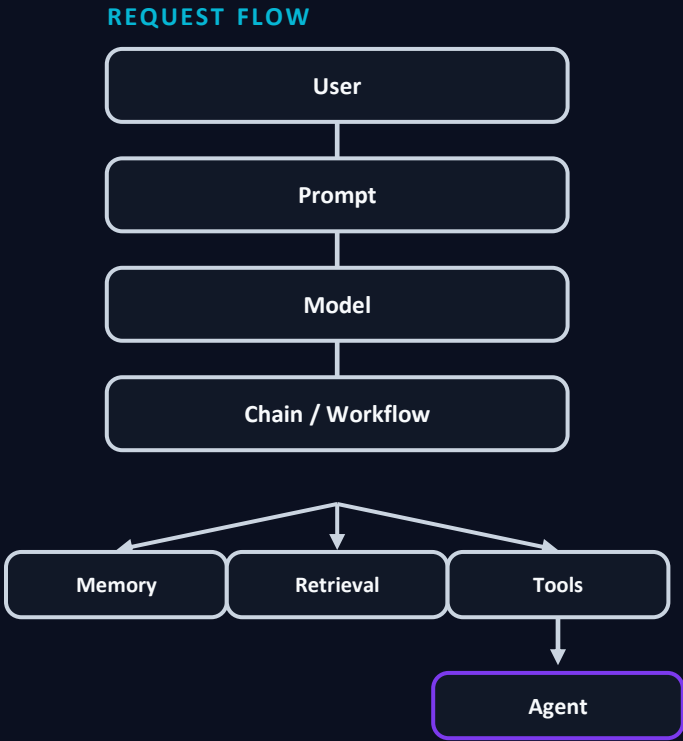
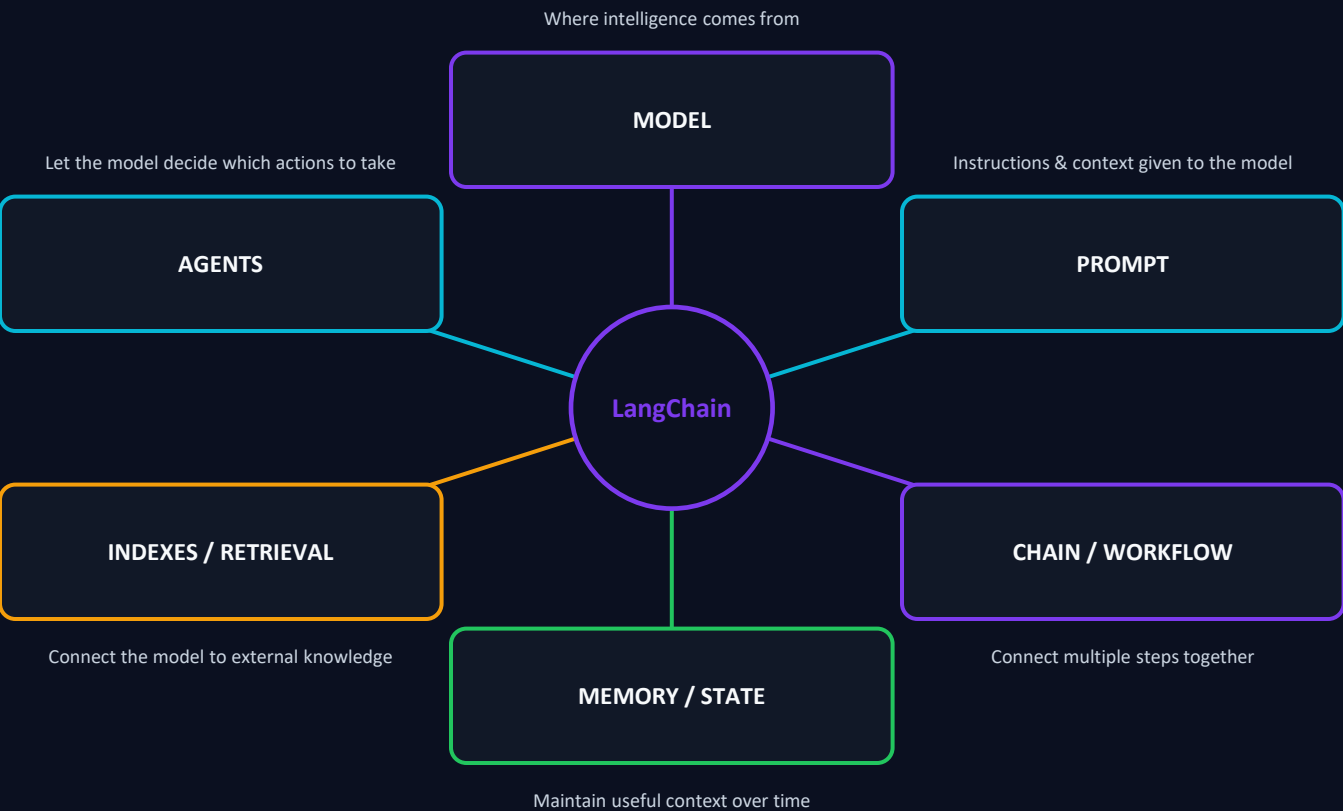
Decide and act using external tools.

LangChain can act as the orchestration layer connecting AI models to application capabilities.

LangChain Is Part of a Larger Ecosystem

FRAMEWORK	PRIMARY FOCUS / TYPICAL USE
LangChain	General LLM application development and integrations
LangGraph	Stateful, controllable agent workflows
lamaIndex	Data ingestion, indexing and retrieval
Haystack	Search, RAG and NLP pipelines
Semantic Kernel	AI orchestration and enterprise integration
AutoGen	Multi-agent application patterns
CrewAI	Role-based multi-agent workflows

The 6 Building Blocks of LangChain

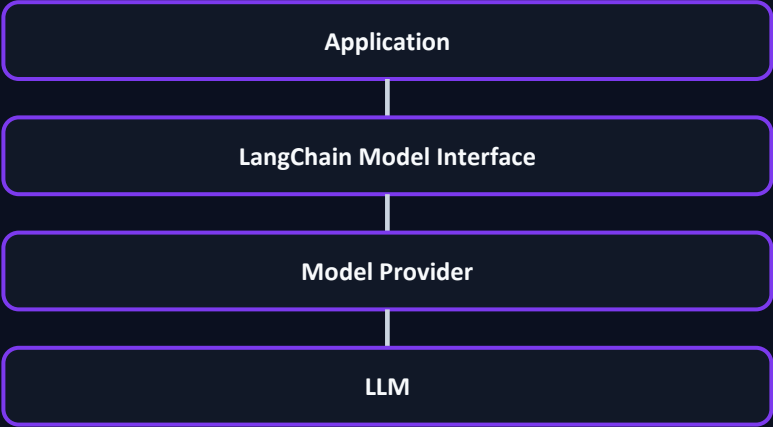


These concepts form the foundation for building practical LangChain applications.

MODEL — The Intelligence Layer

A model is the AI component that processes input and produces an output such as text, structured data, embeddings or decisions.

APPLICATION INTERFACE



CHAT MODELS

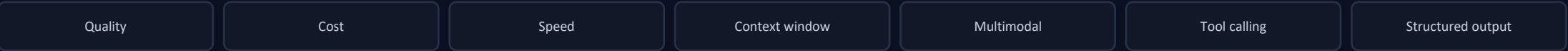
- GPT
- Gemini
- Claude
- Llama

OTHER CAPABILITIES

- Text generation
- Embeddings
- Vision & audio
- Structured output

LangChain provides a consistent application interface while allowing developers to work with different model providers.

SELECTION FACTORS



PROMPT — Communicating With the Model

A prompt is the set of instructions, context and input provided to an AI model to guide its response.



EXAMPLE PROMPT

SYSTEM: You are a professional Python tutor.

CONTEXT: The student is a beginner.

USER: Explain decorators.

CONSTRAINT: Use a simple example.

PROMPT TEMPLATE

Role + Context + Input + Instructions + Output Format

Also: static prompts · dynamic prompts · prompt templates · few-shot examples · structured output instructions

Good prompts provide clear instructions, relevant context and expected output.

CHAINS — Connecting Multiple Steps

A chain / workflow connects multiple operations so the output of one step can become the input of another.

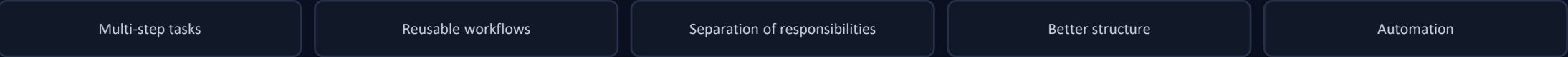
SIMPLE LLM CALL



APPLICATION WORKFLOW



EXAMPLE PIPELINE



Modern LangChain emphasizes composable runnables; complex stateful agent workflows commonly use LangGraph.

4. INDEXES & RETRIEVAL

INDEXES & RETRIEVAL — Connecting AI to Your Data

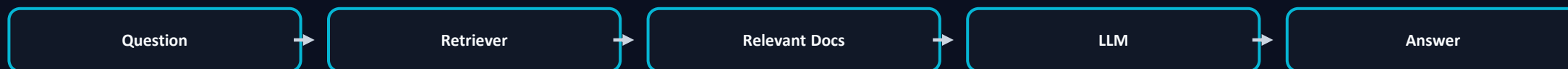
Indexing organizes external information so relevant information can be efficiently retrieved when an AI application needs it.



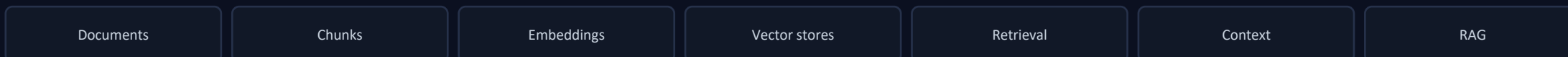
LLM ≠ your private company documents — that's why retrieval exists.

RETRIEVAL-AUGMENTED GENERATION (RAG)

"RAG combines information retrieval with LLM generation."



KEY CONCEPTS



MEMORY / STATE — Maintaining Context

Memory/state allows an AI application to maintain and use relevant information across steps or interactions.

CONVERSATION EXAMPLE

USER: "My name is demo."

AI: "Nice to meet you, demo."

... later ...

USER: "What is my name?"

AI: "Your name is demo."

Modern LangChain apps manage context through message history / state mechanisms; LangGraph provides durable, stateful workflows.

Memory is not one magic database — it's the management of useful context.

WITHOUT CONTEXT



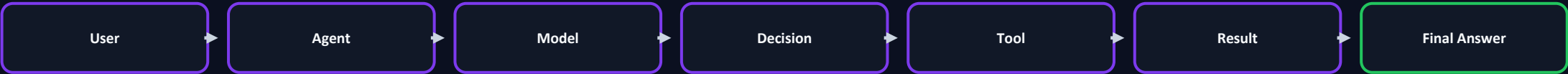
WITH CONVERSATION STATE



- Conversation history
- Short-term context
- Persistent information
- State
- Session-based context

AGENTS — AI That Can Decide What to Do

An AI agent is an application in which a model can determine which actions or tools to use to accomplish a task.



↻ Model re-evaluates after each tool result, until the task is complete

TOOLS AVAILABLE TO THE AGENT



"Find the revenue of company X and calculate its growth."

- Search company data
- Retrieve revenue
- Calculate growth
- Generate explanation

Traditional app: the developer defines every step.

Agentic app: the model selects tools/actions within developer-defined constraints.